



Keep Talking and Nobody Explodes ... UNIX version !

Le principe



Les rôles

Ce jeu se joue à deux joueurs :

- **Le démineur** : c'est celui derrière l'ordinateur.
Lui seul peut interagir avec la bombe.
- **Le gars derrière le fauteuil** : c'est vous !
Vous avez toutes les instructions pour désamorcer cette bombe.

Vous devez coopérer pour comprendre ce que l'un et l'autre doivent faire !

Les modules

Il y a aujourd'hui 7 modules, tous ont pour but de vous faire découvrir des commandes UNIX différentes :

- *files*
- *citations*
- *alphabet*
- *morse*
- *tintin*
- *cities*
- *memory*

Chaque module a une certaine difficulté, et soyez en sûrs : plus vous jouerez, plus vous maîtriserez !

L'ordre dans lequel vous résolvez les modules n'a aucune importance !

⚠ Chaque bombe a un numéro de série propre, appelé `serial`.
Ce fichier se trouve à la racine du projet
(à l'endroit où vous avez lancé la bombe).

Comment gagner ?

Chaque bombe a un niveau de difficulté différents (*voir ci-dessous*). L'unique moyen pour gagner :

Désamorcer tous les modules activés avant la fin du compte à rebours.

Et comment perdre ?

Il y a deux manières de perdre :

- ne pas arriver à désamorcer tous les modules avant la fin du compte à rebours.
- dépasser le nombre d'erreurs maximum autorisé

Chaque fois que vous vous trompez dans un module, une erreur vous serez comptabilisée.

Chaque niveau a un nombre d'erreurs autorisées différents. Ce nombre vous sera donné au début de chaque lancement de bombe. Si vous arrivez au nombre d'erreurs maximum autorisé, la bombe explosera même si vous n'êtes pas à la fin du compteur !

Les commandes

Il y a de nombreuses commandes, et vous seul ayant le manuel peut éclairer le démineur.

- Les commandes de bases sont notés dans l'introduction ci-après.
- Certains modules précisent quelles commandes il faut utilisée pour réussir telle étape
- **Toutes les commandes à utiliser sont listées dans une annexe *Les commandes*.**
Vous y trouverez une explication, des exemples ainsi que la liste des options à connaître.

La progression

Voici la liste des niveaux :

Numéro de niveau	Mini-jeu lancé	Temps	Erreurs autorisées	Votre record	Les modules impliqués
0	fil	50 min	7		N/A
1	citations	50 min	7		N/A
2	fil, citations	50 min	5		N/A
3	alphabet	40 min	5		N/A
4	morse	40 min	5		N/A
5	tintin	40 min	5		N/A
6	cities	40 min	5		N/A
7	memory	40 min	5		N/A
8	alphabet, morse	40 min	4		N/A
9	tintin, cities	40 min	4		N/A
10	memory, citations	40 min	4		N/A
11	2 modules random	30 min	4		
12	3 modules random	30 min	4		
13	3 modules random	30 min	3		
14	4 modules random	30 min	3		
15	3 modules random	30 min	2		
16	4 modules random	30 min	2		
17	3 modules random	20 min	1		
18	3 modules random	20 min	1		
19	5 modules random	30 min	3		
20	6 modules random	30 min	3		
21	Les 7 modules !	30 min	3		
22	Les 7 modules !	20 min	3		
23	Les 7 modules !	10 min	3		
24	Les 7 modules !	10 min	1		

Numéro de niveau	Mini-jeu lancé	Temps	Erreurs autorisées	Votre record	Les modules impliqués
30	1 modules random	10 min	3		
31	2 modules random	10 min	3		
32	3 modules random	10 min	3		
33	4 modules random	10 min	3		
34	5 modules random	10 min	3		
35	6 modules random	10 min	3		
36	Les 7 modules !	10 min	3		

Les commandes de bases à savoir

- Lancer un script `bash` :
`./<nom_script.sh>` - Les scripts `bash` ont généralement une extension en `.sh`
- Lister tous les fichiers : `ls`
- Se déplacer dans un répertoire de fichier :
 - `cd <nom_du_repertoire>` pour entrer dans le répertoire
 - `cd ..` pour remonter au répertoire d'au-dessus
- Pour afficher le contenu d'un fichier : `cat <nom_du_fichier>`

L'auto-complétion

Enfin, il est très important que vous utilisiez l'auto-complétion, en appuyant sur `Tab`. Par exemple, vous voulez ouvrir un fichier qui s'appelle : `YuxiaAsfagSzfqx` ... vous n'allez pas recopier ces caractères un à un !

Vous tapez le début de nom, `Y` et vous appuyez sur `Tab` : cela va vous auto compléter au maximum. (s'il y a un autre fichier commençant par `Y...`, il va s'arrêter au caractère séparant les deux mots).

En appuyant deux fois sur `Tab`, vous afficherez toutes les possibilités d'auto-complétion.

En pratique



Il est possible que des commandes ne soient pas installées sur vos environnements UNIX.

Pour installer une commande, faites :

```
sudo apt-get install <commande>
```

Comment obtenir le jeu ?

1. Lancer un terminal
2. Naviguez dans vos dossiers avec `cd` jusqu'à l'endroit où vous souhaitez télécharger le jeu.
3. Entrer la commande suivante :

```
git clone https://github.com/vrbq/unix-ktane
```

Comment lancer le jeu ?

1. Lancer un terminal.
2. Entrer dans le répertoire principal du jeu, appelé `unix-ktane`
3. Lancer le script `start --level 0` pour démarrer au niveau 0.

(une fois le niveau réussi, passez au niveau 1 !)

Si la bombe a bien été lancée, un message de confirmation apparaîtra sur l'interface.

Comment savoir combien de temps il me reste ?

1. Naviguez jusque dans répertoire principal `unix-ktane`
2. Afficher le contenu du fichier `time` avec la commande `cat`

Module *Fils*



Les commandes qui seront utiles dans ce module

- `cd`
- `ls`
- `cat`
- `rm`

Le principe

Une fois le module lancé, le démineur se trouve face à plusieurs fichiers. Ces fichiers portent un numéro et une couleur. Il peut y avoir entre 3 et 4 fichiers. Chaque numéro est unique, mais la couleur ne l'est pas.

→ Exemple de nom de fichier : `1_rouge.txt`

Suivez les instructions de déminage pour pouvoir **supprimer** le bon fichier.



Pour supprimer un fichier, il faut exécuter la commande `rm` :

```
rm <nom_du_fichier>
```

C'est parti !

Pour lancer le module, lancer le script `start_fils.sh`. Une fois le script lancé, différents fichiers sont apparus dans le répertoire.



Vous ne savez pas comment :

- lancer un script
- afficher le contenu d'un répertoire

⇒ référez vous à la section **Commandes de bases** (première page)

S'il y a exactement 3 fichiers nommé "numéro + couleur" :

- S'il n'y a pas de fichier nommé rouge
⇒ supprimer le deuxième fichier
- Sinon, si le dernier fichier porte la couleur blanche
⇒ supprimer le dernier fichier
- Sinon, s'il y a plus d'un fichier nommé bleu
⇒ supprimer le dernier fichier bleu

- Sinon, supprimer le premier fichier

S'il y a exactement 4 fichiers nommé "numéro + couleur" :

- S'il y a plus d'un fichier nommé rouge, et si le dernier chiffre du numéro de série est impair :
⇒ supprimer le dernier fichier nommé rouge
- Sinon, si le dernier fichier est jaune et s'il n'y a pas de fichier rouge :
⇒ supprimer le premier fichier
- Sinon, s'il y a exactement un fichier nommé bleu
⇒ supprimer le premier fichier
- Sinon, s'il y a plus d'un fichier nommé jaune
⇒ supprimer le dernier fichier
- Sinon, supprimer le deuxième fichier

Comment savoir si ma réponse est correcte ?

Une fois que vous avez identifié le bon fichier, il vous faut le supprimer.

Pour vérifier si votre réponse est correcte, il faut alors lancer le script suivante :

```
verification_fils.sh
```

Si votre réponse est correcte, vous aurez un message du type "Bravo !".

Sinon, vous aurez un message du type "Dommage !".



Vous êtes obligés de relancer le mini-jeu pour soumettre une nouvelle réponse !

Module *Citations*



Les commandes qui seront utiles dans ce module

- `mkdir`
- `mv`
- `cp`
- `vi`

Le principe

Une fois le module lancé, un fichier `citations.txt`. Pour désactiver ce module, le démineur devra effectuer une série de modifications, dictée par vous qui posséder le manuel.

→ Exemple du contenu du fichier `citations.txt` :

1. "Les défis sont ce qui rend la vie intéressante, ..." - Joshua J. Marine
2. "Il n'y a pas d'échec, seulement des retours d'expérience." - Robert Allen
3. "Il ne s'agit pas de savoir si vous êtes capable, ..." - Eric Thomas
4. "Nous ne voyons pas les choses comme elles sont, ..." - Anaïs Nin
5. "Soyez le changement que vous voulez voir dans le monde." - Gandhi
6. "Ce qui compte, ce n'est pas ce que vous regardez, ..." - Henry David Thoreau

Voici la série de modifications à suivre :

1. Créer un répertoire portant un nom prédéfini
2. Copier le fichier `citations.txt` dans ce répertoire
3. Renommer le fichier `citations.txt` en un nom prédéfini
4. Modifier le contenu du fichier `citations.txt`

C'est parti !

Pour lancer le module, lancer le script `start_citations.sh`. Une fois le script lancé, un fichier nommé `citations.txt` est apparu dans le répertoire. Le démineur peut afficher le contenu du fichier avec la commande suivante : `cat <nom_fichier>`



Vous ne savez pas comment :

- lancer un script
- afficher le contenu d'un répertoire

⇒ référez vous à la section **Commandes de bases** (première page)

Création d'un répertoire



Pour créer un répertoire, la commande est : `mkdir <nom_repertoire>`

Exemple : pour créer le répertoire `test` dans le répertoire actuel, voici la commande :

```
mkdir test
```

Pour commencer, il faut créer le répertoire dans lequel devra se trouver une copie du fichier `citations`. Pour connaître comment le répertoire devra s'appeler, suivez les instructions ci-dessous.

Dans le numéro de série ...

- Si le nombre de voyelles est égal à 0, alors le répertoire à créer sera nommé "**burger**".
- Si le nombre de voyelles est égal à 1, alors le répertoire à créer sera nommé "**orange**".
- Dans tous les autres cas (c'est-à-dire quand le nombre de voyelles est égal à 2), le répertoire à créer sera nommé "**pizza**".

Copie du fichier



Pour copier un fichier, la commande est :

```
cp <nom_fichier_original> <nom_repertoire>
```

Exemple : pour déplacer le fichier nommé `original` du répertoire actuel vers le répertoire `dossier`, voici la commande :

```
cp original dossier
```

Renommage du fichier



Pour renommer un fichier, la commande est :

```
mv <nom_fichier> <nouveau_nom>
```

Exemple : pour renommer le fichier nommé `original` en `nouveau` :

```
mv original nouveau
```

Le fichier `citations` désormais déplacé dans le bon répertoire, il faudra le renommer

(sans l'extension `.txt` ! Un fichier n'a pas forcément besoin d'une extension !)

Pour connaître comment le fichier doit être renommé, suivez les instructions ci-dessous :

Dans le numéro de série ...

- Si le nombre de consonnes est égal à 0 **et** que le dernier chiffre du numéro de série est pair, alors le nom du fichier sera **"lion"**.
- Si le nombre de consonnes est égal à 1 **et** que le dernier chiffre du numéro de série est pair, alors le nom du fichier sera **"elephant"**.
- Si le nombre de consonnes est égal à 2 **et** que le dernier chiffre du numéro de série est pair, alors le nom du fichier sera **"koala"**.
- Si le nombre de consonnes est égal à 0 **et** que le dernier chiffre du numéro de série est impair, alors le nom du fichier sera **"panda"**.
- Si le nombre de consonnes est égal à 1 **et** que le dernier chiffre du numéro de série est impair, alors le nom du fichier sera **"kangourou"**.
- Dans tous les autres cas (c'est-à-dire quand le nombre de consonnes est égal à 2 et le dernier chiffre est impair), le nom du fichier sera **"crocodile"**.

Modification du contenu du fichier renommé

Il vous faudra maintenant éditer le fichier copié. Le démoneur connaît les modifications à effectuer car elles sont enregistrées dans un fichier `modifications`. Affichez ce fichier avec la commande `cat`.

Exemple du fichier `modifications` :

Supprimer la ligne qui correspond à l'année de mort de Michael Jackson.

Dans le manuel, vous trouverez une liste de personnalité avec leurs années de naissances et de mort. Seul les deux derniers chiffres de l'année en question sont importants.

Exemple : Michael Jackson est mort en 2009, il faudra supprimer la ligne "09".



La consigne "Supprimer le nom d'auteur" ne signifie pas supprimer le tiret juste avant ;)

Pour éditer un fichier, nous allons utiliser l'éditeur de texte `vi`.

Pour lancer l'éditeur, il faut taper la commande :

```
vi <nom_fichier>
```

Voici quelques astuces pour éditer un fichier sous `vi` :



1. Mode Normal (*déplacement*)

En mode normal, vous pouvez naviguer dans le texte. Pour cela, utiliser les flèches de déplacement.

2. Mode Édition

Pour commencer à éditer du texte, vous devez entrer en mode édition.

- Appuyez sur `i` (pour "insert") lorsque vous êtes en mode normal. Vous verrez le mot "INSERT" apparaître en bas de l'écran, ce qui signifie que vous pouvez maintenant taper du texte.

Pour revenir au mode normal, vous devez appuyer sur la touche `Echap`

3. Supprimer une ligne

Pour supprimer une ligne entière, mettez vous en mode normal et placez vous sur la ligne que vous désirez supprimer. Tapez `dd`. Cela supprimera la ligne où se trouve le curseur.

4. Supprimer un caractère

Pour supprimer un seul caractère, utilisez le mode normal. Placer le curseur sur le caractère que vous souhaitez supprimer et tapez `x`. Cela supprimera le caractère sous le curseur.

5. Sauvegarder et quitter

Pour sauvegarder vos modifications et quitter `vi`, utilisez la commande suivante :

- En mode normal, tapez `:wq` puis appuyez sur `Entrée`. Cela enregistrera les changements et fermera le fichier.

Comment savoir si ma réponse est correcte ?

Une fois que vous avez copié et édité le fichier, vous pouvez vérifier votre réponse en lançant le script `verification_citations.sh`

Si votre réponse est correcte, vous aurez un message du type "Bravo !".
Sinon, vous aurez un message du type "Dommage !".



Vous n'êtes pas obligés de relancer le mini-jeu pour soumettre une nouvelle réponse !

Célébrités (date de naissance et mort)

Nom	Naissance	Mort	Nom	Naissance	Mort
Raphaël	1483	1520	Roland Garros	1888	1918
Louis XIII de France	1601	1643	Clark Gable	1901	1960
Pierre De Fermat	1605	1665	Walt Disney	1901	1966
Rembrandt	1606	1669	Fernandel	1903	1971
Jacques Bernoulli	1654	1705	Jean Gabin	1904	1976
Benjamin Franklin	1706	1790	Christian Dior	1905	1957
Leonhard Euler	1707	1783	Joséphine Baker	1906	1975
Louis XV	1710	1774	Hergé	1907	1983
Jean-Jacques Rousseau	1712	1778	Sœur Emmanuelle	1908	2008
Denis Diderot	1713	1784	Django Reinhardt	1910	1953
Emmanuel Kant	1724	1804	Georges Pompidou	1911	1974
Alexandre Dumas	1802	1870	Alan Turing	1912	1954
Victor Hugo	1802	1885	Rosa Parks	1913	2005
Prosper Mérimée	1803	1870	Hedy Lamarr	1914	2000
George Sand	1804	1876	Edith Piaf	1915	1963
Napoléon III	1808	1873	Roald Dahl	1916	1990
Louis Braille	1809	1852	Henri Salvador	1917	2008
Chopin	1810	1849	Lino Ventura	1919	1987
Franz Liszt	1811	1886	B.B. King	1925	2015
Giuseppe Verdi	1813	1901	Paul Bocuse	1926	2018
Ada Lovelace	1815	1852	Niki De Saint Phalle	1930	2002
James Joule	1818	1889	Yves Saint Laurent	1936	2008
Jules Verne	1828	1905	Thierry Roland	1937	2012
Léon Tolstoï	1828	1910	Johnny Hallyday	1943	2017
Geronimo	1829	1909	Barry White	1944	2003
Eugène Poubelle	1831	1907	Alain Bashung	1947	2009
Emile Zola	1840	1902	Anémone	1950	2019

Nom	Naissance	Mort	Nom	Naissance	Mort
Antonin Dvorak	1841	1904	Robin Williams	1951	2014
Auguste Renoir	1841	1919	Michael Jackson	1958	2009
Buffalo Bill	1846	1917	Diego Maradona	1960	2020
Paul Gauguin	1848	1903	George Michael	1963	2016
Jean Jaurès	1859	1914	Chadwick Boseman	1976	2020
Pierre Curie	1859	1906	Amy Winehouse	1983	2011
Butch Cassidy	1866	1908	Grégory Lemarchal	1983	2007
Raspoutine	1869	1916	Adèle Castillon	2001	

Module *Alphabet*



Les commandes que vous allez apprendre

- la redirection `|` (pipe)
- `ls` et ses options
- `grep`

Le principe

Une fois le module lancé, le démineur se retrouvera devant des dossiers et des fichiers portant différents noms. Pour désamorcer ce mini-jeu, il faudra répondre à la question posée par la bombe ... avec quelques modifications !

C'est parti !

Pour lancer le module, lancer le script `start_alphabet.sh`. Une fois le script lancé, des dossiers contenant des dossiers ou des fichiers apparaissent.



Vous ne savez pas comment :

- lancer un script
- afficher le contenu d'un répertoire

⇒ référez vous à la section **Commandes de bases** (première page)

Un fichier `question` est aussi apparu. Le démineur peut l'afficher avec la commande `cat`. La question posée sera toujours la même :

Quel est le chiffre présent dans le nom du fichier faisant ... octets ?

Pour retrouver ce fichier, quelques commandes utiles :

- `ls` et ses options
- la redirection `|` (pipe)
- `grep`



Aidez le démineur pour trouver la bonne combinaison de commande afin de trouver le plus rapidement possible ce fichier !

Une fois le nom du fichier trouvé, isoler le numéro présent dans le nom du fichier :


```
-rwxrwxrwx 1 root root 429 Oct 28 14:27 file_229.txt
```

Vous devez ensuite transformé ce numéro en fonction du numéro de série. Chaque lettre présente dans le numéro de série est à ajouter à ce numéro. La somme totale sera la réponse à donner dans le script de vérifications.

Voici l'alphabet et les valeurs correspondantes :

Lettre	Nombre associé	Lettre	Nombre associé	Lettre	Nombre associé
A	+12	J	+15	S	-20
B	-3	K	-15	T	+10
C	+25	L	+22	U	+18
D	+17	M	-8	V	-5
E	-10	N	+4	W	+9
F	+8	O	+27	X	+26
G	-1	P	-22	Y	+11
H	+30	Q	+14	Z	-12
I	+6	R	+0		

Comment savoir si ma réponse est correcte ?

Une fois que vous avez retrouvé le numéro à saisir, vous pouvez vérifier votre réponse en lançant le script `verification_alphabet.sh`

Si votre réponse est correcte, vous aurez un message du type "Bravo !".
Sinon, vous aurez un message du type "Dommage !".



Vous n'êtes pas obligés de relancer le mini-jeu pour soumettre une nouvelle réponse !

Module *Morse*



Pour résoudre ce module, vous aurez besoin d'une connexion internet ;)



Les commandes qui seront utiles dans ce module

- `apt-get install`
- `wget`
- `ping`
- la redirection `>` (chevron)
- `ip link`
- `file`
- `du`

Le principe

Le module contient un message en morse, composé de signaux lumineux. Décomposé ce message pour connaître l'instruction à réaliser avant de désamorcer ce module !

C'est parti !



Avant toute chose : il faut installer le paquet `chafa`.
Pour cela, utiliser le gestionnaire de paquet `apt` via la commande suivante :

```
apt-get install <nom_paquet>
```

Une fois le paquet installé, vous pouvez démarrer le module en lançant le script `stant_morse.sh`

Pour trouver le mot à décoder, il faut déchiffrer le code morse du fichier `gif` présent dans le répertoire. Pour cela, tapez la commande : `chafa <nom_fichier_gif>`
→ pour une meilleure lecture en N&B complet, il faut utiliser l'option `-c none`

→ N'hésitez pas à vous servir de l'auto-complétion avec la touche `Tab` !

Comment interpréter

1. Un clignotement court représente un point.
2. Un clignotement long représente un tiret.
3. Il y a une pause entre chaque lettre.
4. Il y a une longue pause avant que le mot ne se répète.

A	• —	U	• • —
B	• • • •	V	• • • —
C	• • • • •	W	• • — •
D	• • • •	X	• • • •
E	•	Y	• • • • •
F	• • • •	Z	• • • • •
G	• • • •		
H	• • • •		
I	• •		
J	• • • • •		
K	• • • •	0	• • • • •
L	• • • •	1	• • • • •
M	• • • •	2	• • • • •
N	• • • •	3	• • • • •
O	• • • •	4	• • • • •
P	• • • •	5	• • • • •
Q	• • • •	6	• • • • •
R	• • • •	7	• • • • •
S	• • • •	8	• • • • •
T	• • • •	9	• • • • •

Alphabet Morse

Le message morse doit donner un mot présent dans la grille ci-dessous.

Mot décodé	Action à réaliser pour désamorcer le module
vitre	Obtenir l'adresse IP du domaine telecom.insa-lyon.fr (commande utile : <code>ping</code>)
ville	Télécharger le fichier présent à cette url : https://tinyurl.com/3xhwvhf2 et donner la taille du fichier en octets (commande utile : <code>wget</code> et <code>du</code> ou <code>ls</code>)
chose	Lancer exactement 3 paquets PING x.com et renvoyer tous ces résultats dans un fichier appelé <code>requete.txt</code> (commande utile : <code>ping</code> et le chevron <code>></code>)
signe	Télécharger le fichier présent à cette url : https://tinyurl.com/3xhwvhf2 et donner la type du fichier (commande utile : <code>wget</code> et <code>file</code>)
linge	Télécharger le fichier présent à cette url : https://tinyurl.com/bdcu59dc et donner la taille du fichier en kilo-octets (commande utile : <code>du</code> ou <code>ls</code>)
ligne	Télécharger le fichier présent à cette url : https://tinyurl.com/496yn2pj et donner la type du fichier (commande utile : <code>wget</code> et <code>file</code>)
champ	Lancer exactement 5 paquets PING google.com et renvoyer tous ces résultats dans un fichier appelé <code>requete.txt</code> (commande utile : <code>ping</code> et le chevron <code>></code>)
litre	Obtenir l'adresse IP du domaine cpe.fr (commande utile : <code>ping</code>)
chaud	Indiquer le nombre d'interfaces réseau présentes sur cette machine (commande utile : <code>ip</code> <code>Link</code>)
bille	Télécharger le fichier présent à cette url : https://tinyurl.com/4czaz2jz et donner la taille du fichier en kilo-octets (commande utile : <code>wget</code> et <code>du</code> ou <code>ls</code>)
balle	Indiquer la valeur de MTU de l'interface réseau loopback <code>lo</code> (commande utile : <code>ip</code> <code>Link</code>)

Une fois le mot déchiffré, réaliser l'action associée pour pouvoir désamorcer le module.

Comment savoir si ma réponse est correcte ?

Pour les mots `champ` et `chose` , il suffit de créer le fichier correspondant et d'activer le script de vérification `verification_morse.sh` .

Pour tous les autres mots, activer le script de vérification. Une question - toujours la même - vous sera posée : **“Quelle est la réponse ?”**. Il vous suffira de donner la réponse précise attendue !

Module *Tintin*



Les commandes qui seront utiles dans ce module

- `ps aux`
- `kill` et ses options
- `htop`

Le principe

Le module va faire défiler des personnages présent dans différents albums de Tintin. A vous de retrouver lequel, et de faire l'action associée !

C'est parti !

Vous pouvez démarrer le module en lançant le script `start_tintin.sh`

Une fois le script lancé, un fichier `personnages` va apparaître. Il se remplira, toutes les 5 secondes, de nom de personnage présent dans l'album concerné. A vous de retrouver le bon album !

NB: si aucun nouveau personnage n'apparaît après 5 secondes, cela veut dire qu'il n'y a plus de personnage à lister ;)

	Tintin	Milou	Dupond et Dupont	Professeur Tournesol	Capitaine Haddock	Nestor	Général Alcazar
Tintin au pays de l'or noir							
Tintin au Tibet							
Tintin Objectif Lune							
Tintin et les Picaros							

En fonction de l'album retrouvé, voici les actions à réaliser :

- Si l'album est **“Tintin au pays de l’or noir”** :

→ Retrouver le PID du processus `sh ./tintin_or_noir.sh`

(commande utile : `ps aux`)

- Si l'album est **“Tintin au Tibet”** :

→ Tuer le processus appelé `sh ./tintin_tibet.sh`

(commande utile : `ps aux` et `kill`)

- Si l'album est **“Tintin Objectif Lune”** :

→ Retrouver le nom de l'utilisateur qui a lancé le processus `sh ./tintin_objectif_lune.sh`

(commande utile : `ps aux`)

- **Si l'album est "Tintin et les Picaros" :**

→ Stopper le processus appelé `sh ./tintin_picaros.sh`

(commande utile : `ps aux` et `kill`)

- **Si vous ne reconnaissez les personnages d'aucun album :**

→ Retrouver combien de "CPU%" le processus `sh ./tintin_etudie_en_TC.sh` consomme

(commande utile : `htop`)

Comment savoir si ma réponse est correcte ?

Lancer le script `verification_tintin.sh` :

- pour les albums "Tintin et les Picaros" et "Tintin au Tibet", la vérification vous dira automatiquement si la bonne action a été réalisée.
- pour les autres, une question sera posée au démineur. A vous d'y répondre selon la question posée au dessus !

Module *Cities*



Les commandes qui seront utiles dans ce module

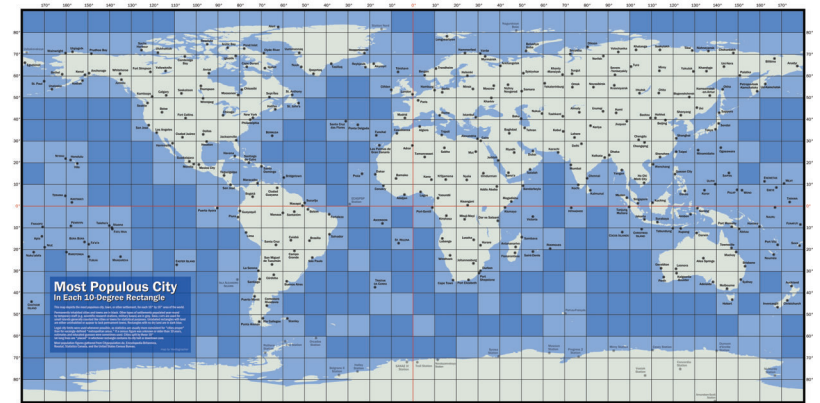
- `pwd`
- `echo $VAR`
- `export`

Le principe

Une fois le script lancé, une série de dossiers appelé par des noms de ville sont créés. A vous de retrouver le dossier à entrer dans la variable d'environnement !

C'est parti !

Après avoir lancé le script `start_cities`, le démineur pourra vous donner des coordonnées GPS qu'il obtiendra dans un fichier appelé `gps_coordinates.txt`. A vous de retrouver la bonne ville !



Vous trouverez une version élargie à la fin de ce manuel !

Pour désamorcer ce module, il faudra guider le démineur pour qu'il :

1. Trouve le répertoire concerné par la ville (commande utile : `find`)
2. Se rendre dans ce dossier avec `cd`
3. Connaître le chemin complet depuis la racine (commande utile : `pwd`)
4. Ajouter ce dossier dans la variable d'environnement `PATH`
→ **afficher une variable d'environnement** : `echo $<nom_variable>` (ex: `echo $PATH`)

Le symbole

`$` permet d'indiquer que c'est une variable. Ici la variable étant globale (variable d'environnement) elle est donc chargée immédiatement.

→ écrire dans une variable d'environnement : `export PATH=$PATH:<votre_nouveau_chemin>`

⚠ Attention à ne pas remplacer la totalité de votre variable `PATH` par votre nouveau chemin !

(ex: `export PATH=$PATH:<votre_nouveau_chemin>` ne conservera pas vos anciens chemins dans votre variable). Cela ne sera pas comptabilisé comme une réponse correcte !

Comment savoir si ma réponse est correcte ?

Vous pouvez vérifier que votre commande a bien marché en affichant de nouveau la variable d'environnement.

Pour vérifier votre réponse, exécutez le script `verification_cities.sh`.

👉 Vous avez besoin de remettre à zéro le module ? Exécutez la commande :

```
source remise_a_zero.sh
```

Module *Memory*



Les commandes qui seront utiles dans ce module

- `tar` et ses options
- `useradd` et `groupadd`
- `chmod`
- `usermod`

Le principe

Des dossiers et des archives vont être créés au lancement du script, ainsi qu'un fichier `image`. Le démineur devra vous faire deviner l'image, afin que vous puissiez lui indiquer les opérations à faire.

Il y a 5 étapes à réussir successivement pour pouvoir désamorcer le module. Le jeu ne vous rappellera pas à quel étape vous êtes .. donc prenez note !

Chaque erreur vous fera recommencer le début du module.

C'est parti !

Lancer le script `start_memory.sh` pour lancer le module.

Voici la liste des images que le démineur pourra vous décrire :

Chaque image correspond à un numéro.
Après avoir reconnu l'image, faites correspondre le numéro de l'étape et le numéro du symbole retrouvé.

3	4	2	1

Étape 1 : Vérification des Dossiers



La commande `tar` (et ses options) sera utile lors de cette étape.

- **Symbole numéro 1** : L'archive `7` doit être désarchivée dans un dossier nommé **basket**.
- **Symbole numéro 2** : L'archive `2` doit être désarchivée dans un dossier nommé **football**.
- **Symbole numéro 3** : L'archive `9` doit être désarchivée dans un dossier nommé **handball**.
- **Symbole numéro 4** : L'archive `2` doit être désarchivée dans un dossier nommé **baseball**.

Étape 2 : Création d'Archives et de l'Utilisateur

- **Symbole numéro 1** : Archiver le dossier `6` en **basket.tar** sans compression (sans `gzip`).

(commande utile : `tar` et ses options)

- **Symbole numéro 2** : L'utilisateur **geronimo** doit être créé.

(commande utile : `useradd`)

- **Symbole numéro 3** : Archiver le dossier `8` en **handball.tar.gz** avec compression `gzip`.

(commande utile : `tar` et ses options)

- **Symbole numéro 4** : Archiver le dossier correspondant au dernier chiffre du Symbole numéro de série (`.serial`) en **rugby.tar.gz** avec compression `gzip`.

(commande utile : `tar` et ses options)

Étape 3 : Vérification des Permissions des Archives



La commande `chmod` sera utile lors de cette étape.

- **Symbole numéro 1** : L'archive `1` doit avoir les droits suivants :
Permissions :
`(rwx)` propriétaire, `(r..x)` groupe, ``(r..)` autres utilisateurs.
- **Symbole numéro 2** : L'archive `3` doit avoir les droits suivants :
Permissions :
`(rw..)` propriétaire, `(rw..)` groupe, `(rw..)` autres utilisateurs.
- **Symbole numéro 3** : L'archive `5` doit avoir les droits suivants :
Permissions :
`(rwx)` propriétaire, `(..x)` groupe, `(..x)` autres utilisateurs.
- **Symbole numéro 4** : L'archive `4` doit avoir les droits suivants :
Permissions :
`(rw..)` propriétaire, `(r..) groupe,` `(r..)` autres utilisateurs.

Étape 4 : Vérification des Groupes et des Droits

- **Symbole numéro 1** : Le propriétaire de l'archive `3` doit être **geronimo**.

(commande utile : `chown`)

- **Symbole numéro 2** : Le groupe **bedonkohe** doit être créé.

(commande utile : `groupadd`)

- **Symbole numéro 3** : L'utilisateur **geronimo** doit être ajouté au groupe **bedonkohe**

(commande utile : `usermod`)

- **Symbole numéro 4** : L'archive dont le numéro correspond au dernier chiffre du numéro de série doit être désarchivée dans un dossier **rugby**.

(commande utile : `tar` et ses options)

Étape 5 : Répétition des Consignes Précédentes

- **Symbole numéro 1** : Refaire l'instruction que vous avez effectué lors de l'étape 1.
- **Symbole numéro 2** : Refaire l'instruction que vous avez effectué lors de l'étape 2.
- **Symbole numéro 3** : Refaire l'instruction que vous avez effectué lors de l'étape 4.
- **Symbole numéro 4** : Refaire l'instruction que vous avez effectué lors de l'étape 3.

Comment savoir si ma réponse est correcte ?

A la fin de chaque étape, vous devez lancer le script `verification_memory.sh`. Cela aboutira :

- soit au passage à l'étape suivante avec le message *“Bravo ! Vous passez à la prochaine étape !”*
- soit à la remise à zéro du module (avec une erreur comptabilisée !)

Vous devrez alors lancer de nouveau le script `start_memory`

Les commandes

Les commandes de bases à savoir

- Lancer un script `bash` :
`./<nom_script.sh>` - Les scripts `bash` ont généralement une extension en `.sh`
- Lister tous les fichiers : `ls`
- Se déplacer dans un répertoire de fichier :
 - `cd <nom_du_repertoire>` pour entrer dans le répertoire
 - `cd ..` pour remonter au répertoire d'au-dessus
- Pour afficher le contenu d'un fichier : `cat <nom_du_fichier>`

La commande `ls` et ses options

La commande `ls` liste les fichiers et répertoires dans le répertoire courant. Voici quelques options importantes :

- `l` : Affiche les détails (permissions, propriétaire, taille).
 - Exemple : `ls -l`
- `a` : Montre tous les fichiers, y compris les cachés.
 - Exemple : `ls -a`
- `h` : Affiche les tailles en format lisible (Ko, Mo).
 - Exemple : `ls -lh`
- `R` : Affiche le contenu de manière récursive (*c-à-d qui parcourt tous les sous-dossiers*).
 - Exemple : `ls -R`
- `t` : Trie par date de modification (plus récent en premier).
 - Exemple : `ls -lt`
- `s` : Trie par taille (plus grand en premier).
 - Exemple : `ls -ls`



Ces options peuvent bien sûr être combinés ! Ex: `ls -lahR`

Création d'un répertoire

Pour créer un répertoire, la commande est : `mkdir <nom_repertoire>`

Exemple : pour créer le répertoire `test` dans le répertoire actuel, voici la commande :

```
mkdir test
```

Copie d'un fichier

Pour copier un fichier, la commande est : `cp <nom_fichier_original> <nom_repertoire>`

Exemple : pour déplacer le fichier nommé `original` du répertoire actuel vers le répertoire `dossier`, voici la commande : `cp original dossier`

Renommer un fichier

Pour renommer un fichier, la commande est : `mv <nom_fichier> <nouveau_nom>`

Exemple : pour renommer le fichier nommé `original` en `nouveau` :

```
mv original nouveau
```

Supprimer un fichier

Pour supprimer un fichier, il faut exécuter la commande `rm` : `rm <nom_du_fichier>`

Installation d'un paquet via `apt`

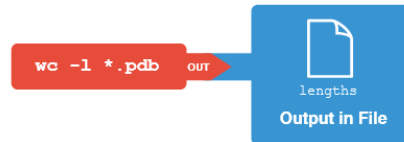
La commande `apt` (Advanced Package Tool) est utilisée pour gérer les paquets dans les distributions basées sur Debian (comme Ubuntu). Elle permet d'installer, de mettre à jour, de supprimer et de rechercher des paquets logiciels.

- Mettre à jour les paquets : `sudo apt update && sudo apt upgrade`
- Installer un paquet : `sudo apt install nom_du_paquet`
- Supprimer un paquet : `sudo apt remove nom_du_paquet`
- Rechercher un paquet : `apt search nom_du_paquet`

Les redirections

Les redirections `>` et `>>` (chevrons)

```
$ wc -l *.pdb > lengths
```

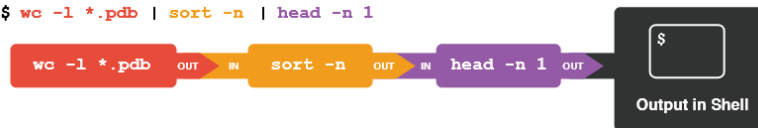


La redirection `|` (pipe)

La redirection avec le symbole

`|`, également appelée "pipe", permet de transmettre la sortie d'une commande comme entrée d'une autre. C'est une fonctionnalité qui permet de combiner plusieurs commandes pour réaliser des opérations complexes sans avoir besoin de créer des fichiers temporaires.

```
$ wc -l *.pdb | sort -n | head -n 1
```



From <https://labordynamicsinstitute.github.io/shell-novice/fig/redirects-and-pipes.svg>

Exemple: On peut utiliser `grep` avec un pipe pour rechercher des mots dans la sortie d'une autre commande. Par exemple, pour rechercher les occurrences de "texte" dans le fichier `fichier.txt` :

```
cat fichier.txt | grep "texte_recherché"
```

Editeur de texte `vi`

1. Mode Normal (*déplacement*)

En mode normal, vous pouvez naviguer dans le texte. Pour cela, utiliser les flèches de déplacement.

2. Mode Édition

Pour commencer à éditer du texte, vous devez entrer en mode édition.

- Appuyez sur `i` (pour "insert") lorsque vous êtes en mode normal. Vous verrez le mot "INSERT" apparaître en bas de l'écran, ce qui signifie que vous pouvez maintenant taper du texte.

Pour revenir au mode normal, vous devez appuyer sur la touche `Echap`.

3. Supprimer une ligne

Pour supprimer une ligne entière, mettez vous en mode normal et placez vous sur la ligne que vous désirez supprimer. Tapez `dd`. Cela supprimera la ligne où se trouve le curseur.

4. Supprimer un caractère

Pour supprimer un seul caractère, utilisez le mode normal. Placer le curseur sur le caractère que vous souhaitez supprimer et tapez `x`. Cela supprimera le caractère sous le curseur.

5. Sauvegarder et quitter

Pour sauvegarder vos modifications et quitter `vi`, utilisez la commande suivante :

- En mode normal, tapez `:wq` puis appuyez sur `Entrée`. Cela enregistrera les changements et fermera le fichier.
 - Si vous ne voulez pas sauvegarder, tapez `:q!`

La commande `grep`

La commande

`grep` (*Global Regular Expression Print*) est un outil puissant pour rechercher des chaînes de caractères dans des fichiers ou dans le texte d'entrée. Elle est souvent utilisée dans le traitement de texte ou le filtrage de logs. Voici un exemple typique d'utilisation de `grep` :

1. Rechercher dans un fichier :

```
grep "texte" fichier.txt
```

La commande **ps aux**

La commande **ps aux** affiche la liste complète des processus en cours d'exécution sur le système, avec des détails comme l'identifiant du processus (PID), l'utilisateur ainsi que la commande lancée.

```
ps aux | grep "nom_du_processus"
```

La commande **kill**

La commande **kill** permet d'envoyer des signaux aux processus, souvent pour les arrêter. Par défaut, **kill** envoie le signal **TERM** pour demander l'arrêt d'un processus.

```
kill <PID>
```

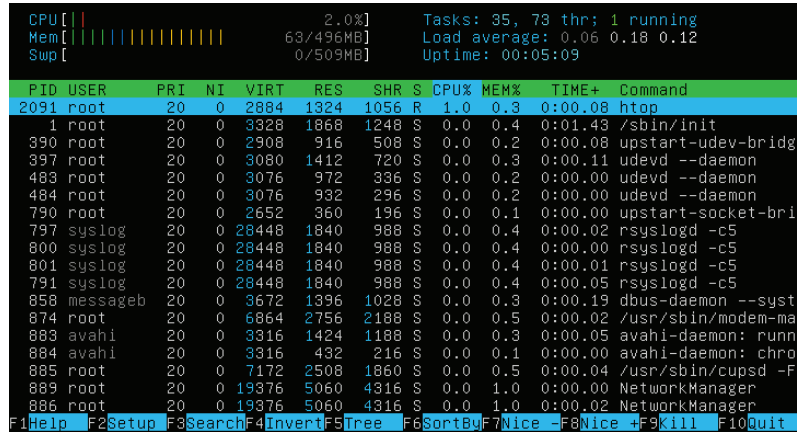
Les options utiles :

- **-9** : Envoie le signal **KILL** pour forcer l'arrêt immédiat si le processus ne répond pas au signal **TERM**.
- **-l** : Liste les signaux disponibles.
- **-STOP** : Stopper un processus
- **-CONT** : Reprendre un processus arrêté

La commande **htop**

htop est un outil interactif pour surveiller en temps réel les processus et l'utilisation des ressources (CPU, RAM, swap) sur un système Linux. Il fournit une interface colorée et plus intuitive que **top**.

Il suffit de lancer la commande **htop** pour voir cette fenêtre apparaître.



Pour en savoir plus sur les différentes colonnes, vous pouvez vous renseigner avec `man htop`.

La commande **find**

La commande **find** permet de rechercher des fichiers et répertoires dans l'arborescence de fichiers Unix en fonction de divers critères.

Syntaxe de base

```
find [chemin] [critères] [actions]
```

- **chemin** : Répertoire de recherche (par défaut, **.** pour le répertoire actuel)
- **critères** : Critères pour filtrer les résultats (nom, taille, type, etc.)
- **actions** : Actions à effectuer sur les fichiers trouvés (afficher, supprimer, exécuter une commande, etc.)

Options courantes

Option	Description	Exemple
-name	Cherche par nom exact (case-sensitive).	<code>find . -name "fichier.txt"</code>
-iname	Cherche par nom (insensible à la casse).	<code>find . -iname "fichier.txt"</code>
-type	Filtre par type de fichier (f pour fichier, d pour dossier).	<code>find . -type f</code>
-size	Cherche par taille.	<code>find . -size +10M</code> (plus de 10 Mo)
-mtime	Fichiers modifiés il y a N jours.	<code>find . -mtime -7</code> (moins de 7 jours)
-user	Fichiers appartenant à un utilisateur.	<code>find / -user username</code>
-exec	Exécute une commande sur les résultats.	<code>find . -name "*.log" -exec rm {} \;</code>
-delete	Supprime les fichiers trouvés (utilisé avec précaution).	<code>find . -name "*.tmp" -delete</code>
-empty	Recherche les fichiers ou dossiers vides.	<code>find . -empty</code>
-perm	Fichiers avec des permissions spécifiques.	<code>find . -perm 644</code>
-maxdepth	Limite la profondeur de recherche.	<code>find . -maxdepth 1 -name "*.txt"</code>

La commande `ping`

La commande `ping` sous Unix est utilisée pour tester la connectivité réseau entre l'hôte local et une adresse IP ou un nom de domaine. Elle envoie des paquets ICMP (Internet Control Message Protocol) à la cible et attend des réponses, mesurant le temps aller-retour pour chaque paquet. Cette commande est utile pour diagnostiquer les problèmes de réseau, vérifier si une machine est en ligne, ou évaluer la latence. Sa syntaxe de base est :

```
ping [options] <destination>
```

Voici les options les plus courantes :

Option	Description	Exemple
<code>-c</code>	Spécifie le nombre de paquets à envoyer	<code>ping -c 4 google.com</code>
<code>-i</code>	Définit l'intervalle en secondes entre les paquets	<code>ping -i 2 google.com</code>
<code>-s</code>	Définit la taille des paquets envoyés en octets	<code>ping -s 128 google.com</code>
<code>-t</code>	Définit le TTL (Time To Live), le nombre de sauts maximum	<code>ping -t 64 google.com</code>
<code>-W</code>	Spécifie le délai d'attente pour une réponse en secondes	<code>ping -W 2 google.com</code>
<code>-q</code>	Mode silencieux, affiche uniquement le résumé	<code>ping -c 4 -q google.com</code>
<code>-f</code>	Envoie les paquets de manière continue (mode flood, root)	<code>ping -f google.com</code>
<code>-n</code>	Affiche les adresses en format numérique (pas de résolution DNS)	<code>ping -n google.com</code>

La commande `pwd`

La commande `pwd` (*Print Working Directory*) affiche le chemin absolu du répertoire de travail actuel.

```
pwd [options]
#Sortie : /home/utilisateur/projet
```

La commande `export`

La commande `export` en Unix sert principalement à définir des variables d'environnement, c'est-à-dire des variables disponibles pour le shell actuel et pour tous les processus lancés depuis ce shell. Elle est souvent utilisée dans les scripts pour s'assurer que certaines variables soient accessibles dans les sous-processus.

- Pour exporter une variable : `export VARIABLE=valeur`
- Pour ajouter une valeur à une variable déjà existante : `export VARIABLE=$VARIABLE:valeur`
- Pour lister toutes les variables exportées dans l'environnement actuel : `export -p`

Mais à quoi servent les variables d'environnement ?

1. Configuration du système :

- Elles permettent de définir des paramètres globaux qui influencent le comportement des programmes et des scripts. Par exemple, la variable `PATH` indique au système où chercher les exécutables.

2. Personnalisation de l'environnement utilisateur :

- Les utilisateurs peuvent personnaliser leur expérience en modifiant des variables telles que `HOME` (répertoire personnel) ou `LANG` (langue de l'interface).

3. Transmission d'informations entre processus :

- Les variables d'environnement permettent aux processus de partager des informations. Par exemple, une application peut lire une clé API à partir d'une variable d'environnement pour accéder à un service.

4. Contrôle des configurations des applications :

- De nombreuses applications utilisent des variables d'environnement pour configurer leur comportement sans nécessiter de fichiers de configuration spécifiques. Par exemple, une application web pourrait utiliser une variable pour définir l'URL de la base de données.

5. Gestion des ressources système :

- Les variables peuvent également contrôler les ressources utilisées par les processus, comme la mémoire ou le nombre de threads.

▼ Exemples de variables d'environnement courantes

- `PATH` : Chemins des répertoires à rechercher pour les exécutables.
- `HOME` : Répertoire personnel de l'utilisateur.
- `USER` : Nom de l'utilisateur courant.
- `SHELL` : Chemin vers le shell par défaut de l'utilisateur.
- `LANG` : Langue et paramètres régionaux de l'utilisateur.

La commande **tar**

La commande **tar** en Unix est utilisée pour créer, extraire et manipuler des archives de fichiers (souvent appelées "tarballs"). Elle est couramment utilisée pour combiner plusieurs fichiers et dossiers dans un seul fichier compressé. La syntaxe générale est la suivante :

```
tar [options] archive.tar [fichiers ou dossiers]
```

Option	Description	Exemple
-c	Crée une nouvelle archive	tar -cvf archive.tar dossier/
-x	Extrait le contenu d'une archive	tar -xvf archive.tar
-v	Mode verbeux, affiche les fichiers traités	tar -cvf archive.tar dossier/ (avec -v pour afficher)
-f	Spécifie le nom de l'archive	tar -cvf archive.tar dossier/
-z	Comprime avec gzip	tar -czvf archive.tar.gz dossier/
-j	Comprime avec bzip2	tar -cjvf archive.tar.bz2 dossier/
-t	Liste le contenu d'une archive sans extraire	tar -tvf archive.tar
-C	Change de répertoire avant l'extraction	tar -xvf archive.tar -C /chemin/destination
-p	Préserve les permissions des fichiers	tar -cvpf archive.tar dossier/
--exclude	Exclut des fichiers ou dossiers spécifiques	tar -cvf archive.tar dossier/ --exclude="*.log"

Ajout d'utilisateur (**useradd**) ou de groupe (**groupadd**)

La commande **useradd** crée un nouvel utilisateur en ajoutant une entrée dans les fichiers système appropriés. Voici la syntaxe de base :

```
useradd [options] nom_utilisateur
```

La commande **groupadd** permet de créer un nouveau groupe d'utilisateurs. La syntaxe basique est :

```
groupadd [options] nom_groupe
```

Voici les options les plus courantes, avec des exemples d'utilisations :

Commande	Option	Description	Exemple
useradd	-m	Crée un répertoire personnel pour l'utilisateur	useradd -m nouvel_utilisateur
useradd	-d /chemin	Spécifie un répertoire personnel personnalisé	useradd -d /home/custom nouvel_utilisateur
useradd	-s /bin/shell	Définit le shell par défaut de l'utilisateur	useradd -s /bin/zsh nouvel_utilisateur
useradd	-g groupe	Associe l'utilisateur à un groupe initial	useradd -m -g groupe_initial nouvel_utilisateur
useradd	-G groupe1,groupe2	Associe l'utilisateur à des groupes supplémentaires	useradd -G dev,admin nouvel_utilisateur
useradd	-p mot_de_passe_chiffré	Définit un mot de passe chiffré pour l'utilisateur	useradd -p \$(openssl passwd -1 mot_de_passe) nouvel_utilisateur
useradd	-u UID	Définit un UID spécifique pour l'utilisateur	useradd -u 1050 nouvel_utilisateur
groupadd	-g GID	Définit un GID spécifique pour le groupe	groupadd -g 2001 nouveau_groupe
groupadd	-f	Évite les erreurs si le groupe existe déjà	groupadd -f existant_groupe

Ajout d'un utilisateur dans un groupe (**usermod**)

La commande **usermod** en Unix est utilisée pour modifier les paramètres d'un utilisateur existant. Elle permet d'ajuster divers attributs, comme le groupe primaire, le dossier personnel, ou la date d'expiration du compte. La commande nécessite souvent des privilèges super utilisateur (**sudo**), car elle peut apporter des modifications importantes sur les utilisateurs du système.

Option	Description	Exemple
-aG	Ajoute l'utilisateur à des groupes supplémentaires sans enlever les existants.	sudo usermod -aG groupe1,groupe2 utilisateur
-c	Définit ou modifie le commentaire de l'utilisateur.	sudo usermod -c "Nouveau commentaire" utilisateur
-d	Modifie le répertoire personnel de l'utilisateur.	sudo usermod -d /nouveau/chemin utilisateur

-e	Définit la date d'expiration du compte (format AAAA-MM-JJ).	sudo usermod -e 2025-12-31 utilisateur
-G	Ajoute l'utilisateur à des groupes supplémentaires (séparés par des virgules).	sudo usermod -G groupe1,groupe2 utilisateur
-l	Change le nom de l'utilisateur.	sudo usermod -l nouveau_nom utilisateur
-L	Verrouille le compte utilisateur (empêche la connexion).	sudo usermod -L utilisateur
-p	Définit le mot de passe de l'utilisateur (hashé).	sudo usermod -p hash_du_mot_de_passe utilisateur
-s	Change le shell par défaut de l'utilisateur.	sudo usermod -s /bin/bash utilisateur
-u	Change l'UID de l'utilisateur.	sudo usermod -u 1234 utilisateur
-U	Déverrouille un compte utilisateur verrouillé.	sudo usermod -U utilisateur

Appartenance d’un fichier

Comment afficher ces permissions ?

Pour afficher les droits de fichiers ou dossiers, utilisez la commande `ls -l`. Cela affichera les permissions sous forme symbolique pour chaque fichier ou dossier listé.

```
-rwxr-xr--  1 user group 4096 Nov  3 12:00 fichier.txt
```

La première colonne (par exemple, `-rwxr-xr--`) représente les permissions :

- Le **premier caractère** indique le type d’élément :
 - `-` : fichier
 - `d` : dossier
 - `l` : lien symbolique
- Les **neuf caractères suivants** se lisent par groupes de trois, représentant les permissions pour l'utilisateur, le groupe et les autres, respectivement :
 - `r` : lecture
 - `w` : écriture
 - `x` : exécution
 - `-` : absence de permission

Dans l'exemple `-rwxr-xr--` :

- `rwx` : l'utilisateur a les permissions de lecture, écriture, et exécution.
- `r-x` : le groupe a les permissions de lecture et exécution.
- `r--` : les autres ont uniquement la permission de lecture.

Comment les changer ?

En Unix, les commandes `chmod`, `chown` et `chgrp` permettent de gérer les permissions et les propriétaires des fichiers et dossiers.

- `chmod` : Modifie les permissions d'accès pour les utilisateurs, groupes et autres. Les permissions sont représentées par trois catégories : lecture (`r`), écriture (`w`) et exécution (`x`), chacune pouvant être attribuée aux trois types d'utilisateurs.
- `chown` : Change le propriétaire d'un fichier ou d'un dossier. Cette commande est souvent utilisée pour attribuer un fichier à un utilisateur ou un groupe spécifique.
- `chgrp` : Change le groupe propriétaire d'un fichier ou d'un dossier. Elle permet de modifier uniquement le groupe sans affecter le propriétaire.

Voici un tableau des options courantes pour chacune de ces commandes.

Commande	Option	Description	Exemple
chmod	u+x	Ajoute la permission d'exécution pour l'utilisateur	chmod u+x fichier
chmod	g-w	Retire la permission d'écriture pour le groupe	chmod g-w fichier
chmod	o+r	Ajoute la permission de lecture pour les autres	chmod o+r fichier
chmod	755	Définit les permissions pour utilisateur/groupe/autres <i>(détails sur le numéro ci-dessous)</i>	chmod 755 fichier
chmod	-R	Applique les changements de façon récursive	chmod -R 755 dossier
chown	user:group	Change le propriétaire et le groupe	chown user:group fichier
chown	user	Change uniquement le propriétaire	chown user fichier
chown	-R	Applique le changement de façon récursive	chown -R user dossier
chgrp	group	Change le groupe propriétaire	chgrp group fichier
chgrp	-R	Applique le changement de façon récursive	chgrp -R group dossier

La commande `chmod XXX`

La commande `chmod XXX` permet de définir les permissions d'un fichier ou dossier en utilisant une notation numérique, où chaque chiffre de la séquence `xxx` représente les permissions pour l'utilisateur, le groupe, et les autres, respectivement.

Les chiffres sont calculés en additionnant les valeurs suivantes pour chaque type de permission :

- Lecture (r)** : 4
- Écriture (w)** : 2
- Exécution (x)** : 1

Par exemple :

- 7** (4+2+1) : lecture, écriture et exécution.
- 6** (4+2) : lecture et écriture.

- **5** (4+1) : lecture et exécution.
- **4** : lecture seulement.

Exemples courants

- `chmod 755 fichier` : L'utilisateur a toutes les permissions (7), le groupe et les autres peuvent lire et exécuter (5).
- `chmod 644 fichier` : L'utilisateur peut lire et écrire (6), tandis que le groupe et les autres ne peuvent que lire (4).

Comment choisir les numéros ?

1. **Déterminez les permissions pour chaque type d'utilisateur** (utilisateur, groupe, autres).
2. **Calculez la somme** des valeurs de permissions pour chaque catégorie.
3. **Formez la séquence xxx** en plaçant chaque résultat dans l'ordre.

Ainsi, `chmod 755 fichier` donnerait toutes les permissions à l'utilisateur, et seulement lecture et exécution pour les autres.